

Intro to Behavior-Driven Design & User Stories

ESaaS §7.1

© 2023 Armando Fox, David Patterson, Michael Ball
Licensed under Creative Commons Attribution-
NonCommercial-4.0 Unported License



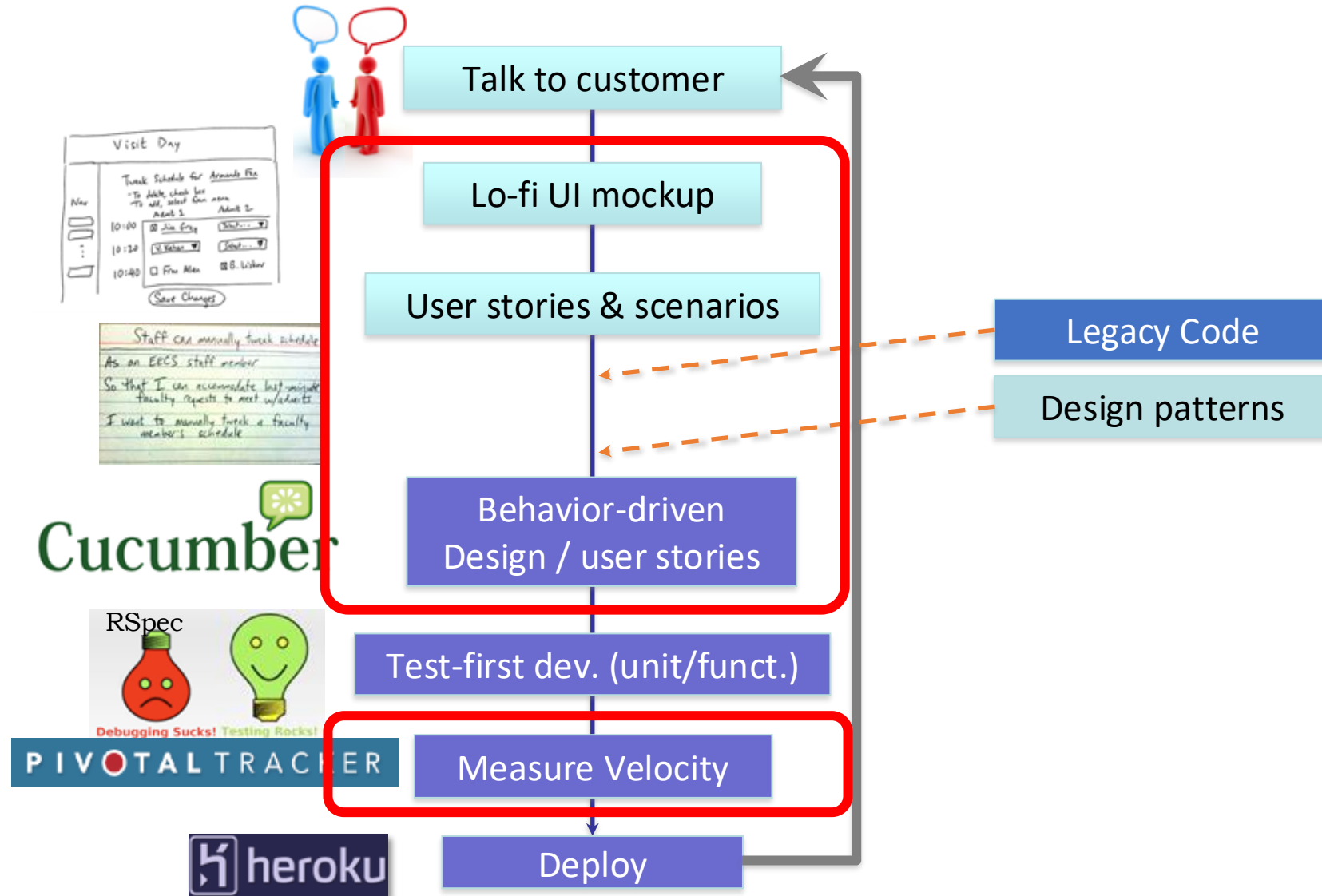
How does Agile avoid failure?

- Work closely, continuously with **stakeholders** to develop requirements, tests
 - Why? One reason SW projects fail is they don't do what customer wants
 - Other stakeholders: end users, developers, maintenance programmers, operators, project managers, ...
- Maintain working prototype while deploying new features every **iteration** (~1-2 weeks, vs. months)
 - Why? Customer can quickly try new features and speak up if on the wrong track (validation vs. verification)

Behavior-Driven Design (BDD)

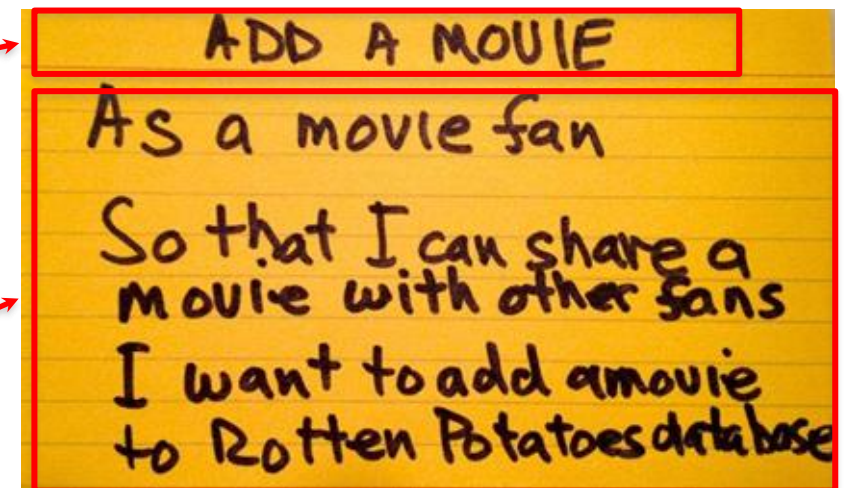
- Asks questions about desired behavior of app *before and during development* to reduce miscommunication
- Captures answers as *user stories*
- Concentrates on *behavior* of app vs. *implementation* of app
 - Test Driven Development or TDD (future segments) tests implementation
- “steals great ideas from HCI”

Reminder: Agile Iteration



User Stories (from HCI!)

- 1-3 sentences in everyday language
 - Fits on an index card
 - Written by/with customer
- “Connextra” format:
 - Feature name
 - **As a** [kind of stakeholder],
So that [I can achieve some goal],
I want to [do some task]
 - 3 phrases must be there, can be in any order
- Idea is to capture “business value” of feature



Why small cards & plain language?

- Plain language is nonthreatening to nontechnical customers
 - all stakeholders help brainstorm
- Easy to rearrange
 - all stakeholders help prioritize
- Short and specific to one behavior
 - easier to change during development

“Behavior” is in the eye of the stakeholder

- *Moviegoer can see which of their friends are going to a show*
 - As a moviegoer
 - So that I can enjoy the movie with my friends
 - I want to see which of my social network friends are attending a given movie
- *Theater manager can see patron’s connected social network (“friends”)*
 - As a box office manager
 - So that I can induce a patron to buy a ticket
 - I want to show them which of their friends are going to the show

SMART User Stories

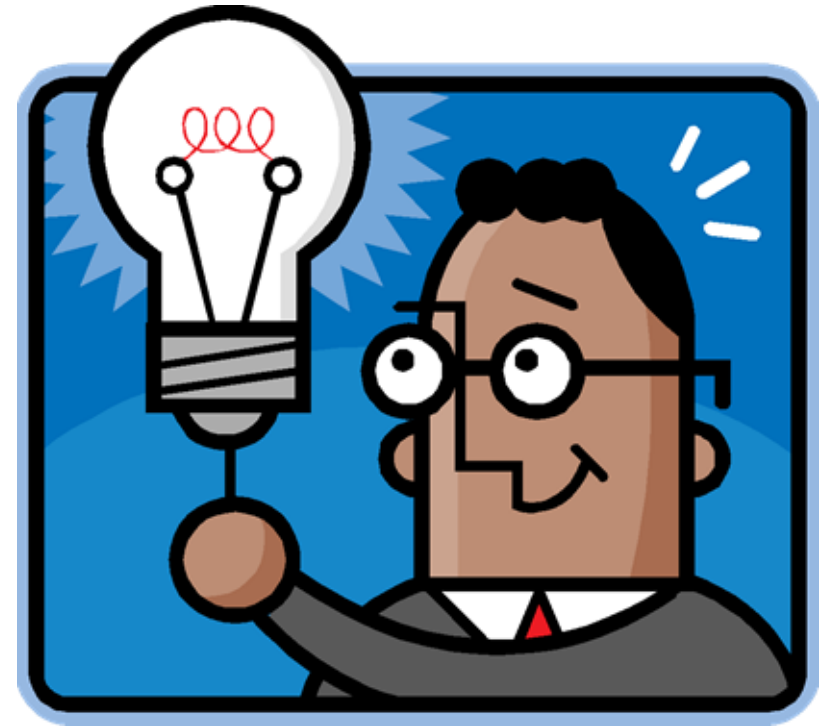
ESaaS §7.2

© 2023 Armando Fox, David Patterson, Michael Ball
Licensed under Creative Commons Attribution-
NonCommercial-4.0 Unported License



What makes a good user story?

- **S**pecific
- **M**easurable
- **A**chievable
(ideally, implement in 1 iteration)
- **R**elevant (“the 5 whys”)
- **T**imeboxed (know when to give up)



Specific & Measurable

- Idea: can you write down a step-by-step procedure that someone with no knowledge of the app could do to verify the specific behavior?
- Anti-example: “UI should be more user-friendly”
- Example: Given/When/Then.
Given some very specific starting condition(s),
When I take specific action X,
Then one or more specific thing(s) should happen
- In general, *you could write an automated test for it*

Achievable

- Complete in 1 iteration
 - Otherwise: divide into multiple stories that form an *epic*
 - Goal: Always have working code at end of iteration
 - <1 story per iteration ☐ need to improve point estimation!
- Related: **Timeboxed**

Relevance: the 5 Whys

“Show patron’s social-network friends”

As a box office manager

So that I can induce a patron to
buy a ticket

I want to show them which
friends are going to a given show

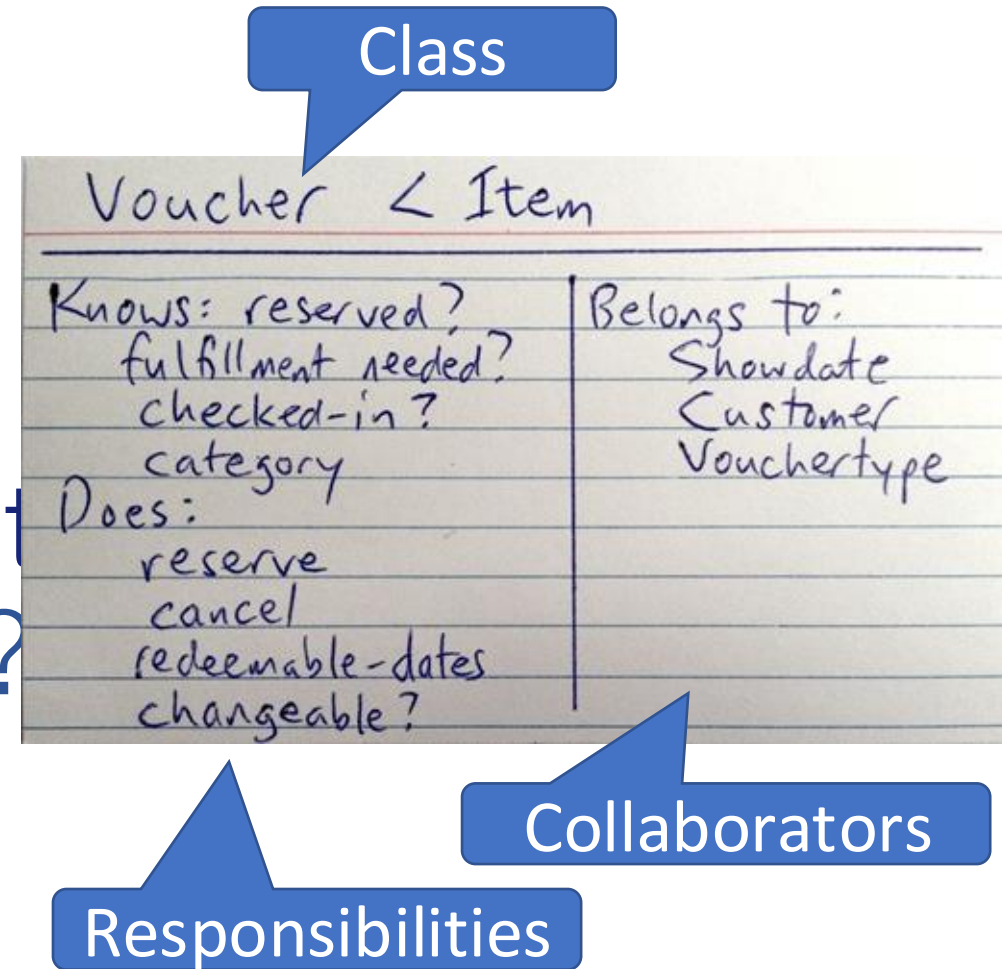


SMART or not?

- User can search for a movie by title
- Given I have a free movie pass, I want to redeem it for an eligible movie before it expires
- When adding a movie, 99% of page loads should appear within 3 seconds
- note, we haven't talked about business value here...

Class-Responsibility-Collaborator (CRC) Cards

- What does this class **know**?
- What does it **do**?
- What other classes does it **collaborate with** and how?
- Goal: make the design concrete



Lo-Fi UI Sketches & Storyboards

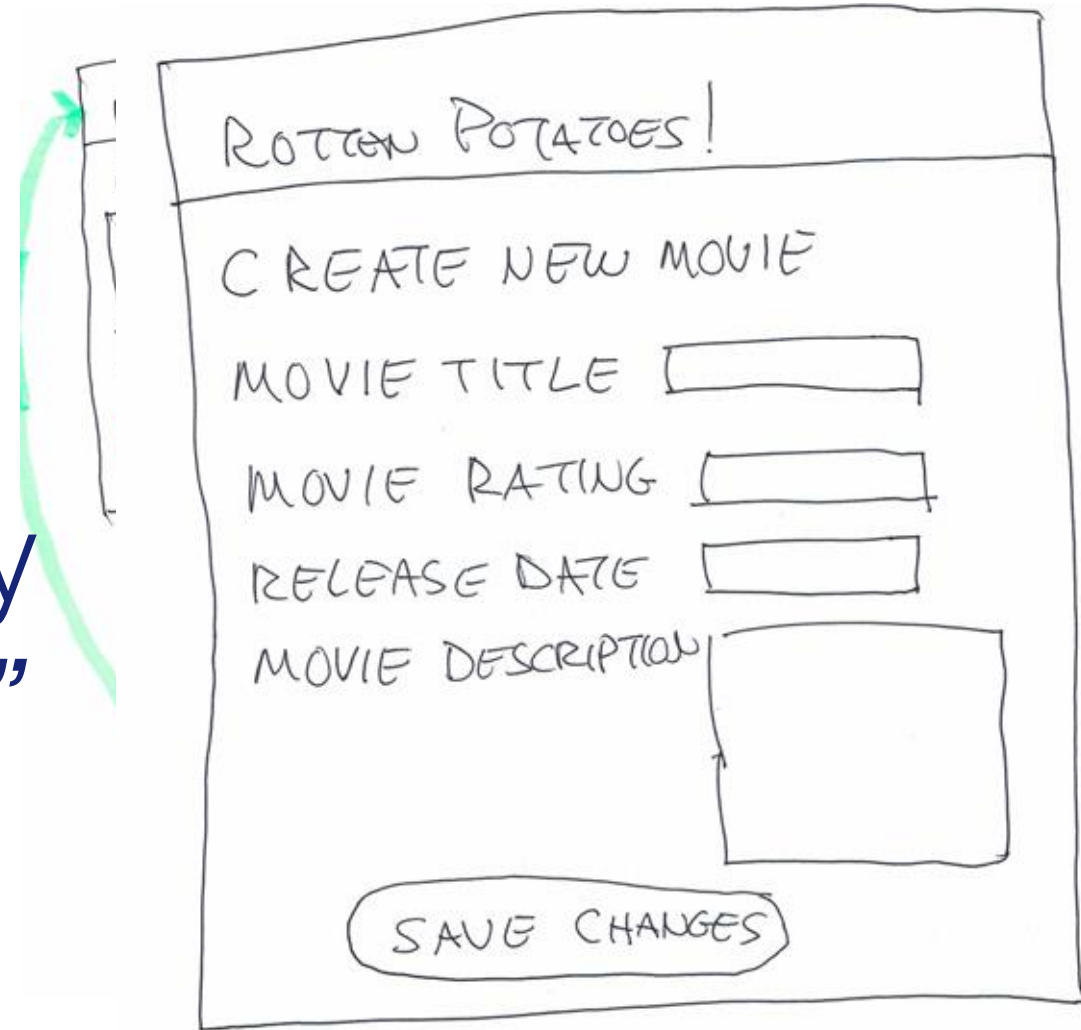
ESaaS §7.3

© 2023 Armando Fox, David Patterson, Michael Ball
Licensed under Creative Commons Attribution-
NonCommercial-4.0 Unported License



Avoiding WISBNWIW*: Customer participates in UI development

- Pen & paper sketch!
 - Why not wireframe?
- **Storyboards** animate the prototype manually
 - *similar to “Wizard of Oz” testing in HCI*



* “What I said but not what I wanted”

Why lo-fi?

- Less intimidating to nontechnical stakeholders
- More likely to suggest changes to UI if no code behind it
- More likely to focus on *interaction (behavior)* rather than colors, fonts, ...
 - CSS can make it look nice later

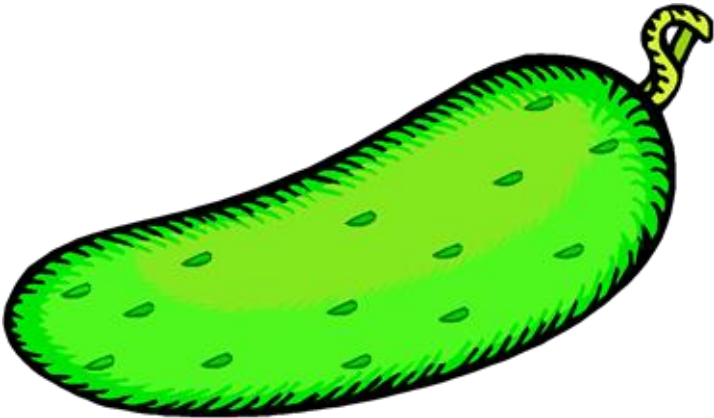
#8. Working with the customer: BDD & Lo-Fi Prototyping



- “Lo-fi and storyboards really helpful in working with customer”
- “Frequent customer feedback is essential”
- “What we thought would be cool is not what customer cared about”
- “We did hi-fi prototypes, and invested a lot of time only to realize customer didn’t like it”
- “Never realized how challenging to get from customer description to technical plan”

From Stories to Acceptance Tests: Introducing Cucumber

ESaaS §7.6



© 2023 Armando Fox, David Patterson, Michael Ball
Licensed under Creative Commons Attribution-
NonCommercial-4.0 Unported License



Key idea #1: story ⇔ acceptance test

- What if you could create a test from customer-friendly user story?
 - Acceptance: ensure satisfied customer
 - Integration: test full-stack functionality
- Cucumber connects “almost plain language” test description to code
 - Plain language ☐ can discuss with customer
 - You define your own “vocabulary” (a *domain language*) & terms that make sense for your app

5 Step Keywords

Given state of world before event: set up preconditions

When user does something

- e.g., simulate user pushing a button

Then check expected postconditions

And & But extend previous step

Example User Story

Feature: `User can manually add movie` Feature (name of story)

Scenario: `Add a movie` ≥1 Scenarios / Feature (happy/sad paths)

Given I am on the RottenPotatoes home page

When I follow "Add new movie"

Then I should be on the Create New Movie page

When I fill in "Title" with "Men In Black"

And I select "PG-13" from "Rating"

And I press "Save Changes"

Then I should be on the RottenPotatoes home page

And I should see "Men In Black"

3 to 8 Steps / Scenario

Key Idea #2: Regexes match scenario steps to Step Definitions ("step defs")

When /I follow "(.*)"/ do |name|
 ...code that triggers app to follow link
end

args matches # of
capture groups

When I follow "Add new movie"

- gem cucumber-rails-training-wheels provides some very basic step defs...

From Stories to Acceptance Tests: Introducing Capybara

ESaaS §7.6



© 2023 Armando Fox, David Patterson, Michael Ball
Licensed under Creative Commons Attribution-
NonCommercial-4.0 Unported License



“Fake User” to try Scenarios

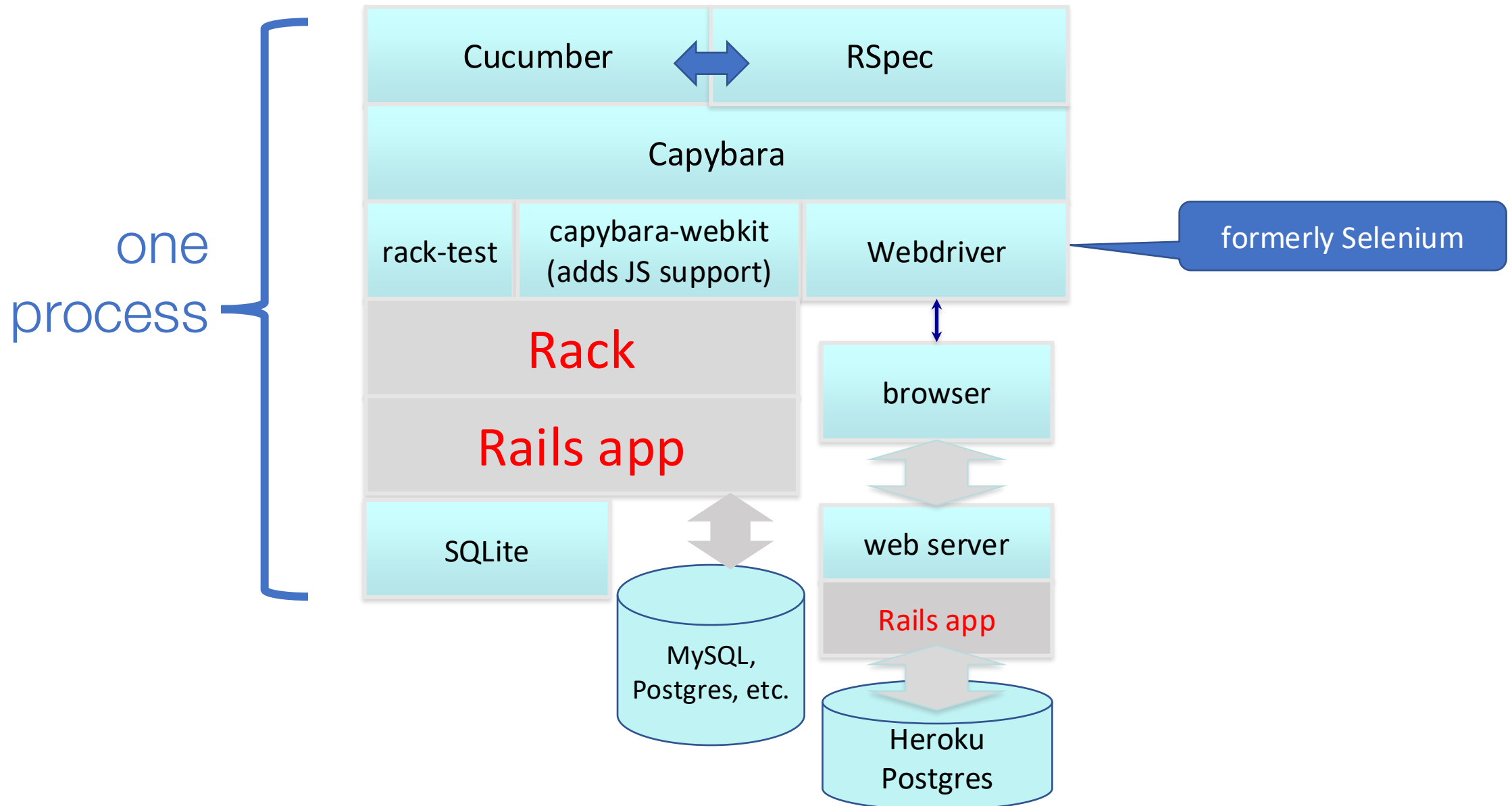
- Capybara library simulates user by automating browser
 - HTTP request to any route in app
 - Submit forms, interact with UI elements
 - Interact with app, receive pages
 - Execute JavaScript
 - Parse HTML, allow programmatic inspection



A (very) few Capybara methods

```
visit "/movies"  
click_link "Add new movie"  
fill_in "Title", :with => "The Thing"  
fill_in "form#form1 nameField", :with => ...  
select "R" from "Ratings"  
click_button "submit"
```

Rails/SaaS testing stacks



Points & Velocity

ESaaS §7.4

© 2023 Armando Fox, David Patterson, Michael Ball
Licensed under Creative Commons Attribution-
NonCommercial-4.0 Unported License



Some stories harder than others...

- Vote: 1...3 points for story (else: Epic)
 - Usually, >3 points means “I don’t know”
- **Velocity**: moving average of #points delivered per iteration



Use within team, not across teams!



Opinion: 1-point stories are good!

Goal: improve estimation ability over time

Real systems have 100s of stories...

- *Backlog*: stories prioritized but not yet started (“in the queue”)
 - Prioritize w/customer according to business value
- *Icebox*: stories not yet pointed/prioritized
 - may or may not get done later
- *Spike*: short investigation of problem, technique, tool, library, etc.
 - bound the time
 - when done, discard code, then do it properly

Centralizing a Team's Activities...

- Features: SMART business value and have points
- Bugs: aren't worth points
- Chores: necessary but don't have business value or points (eg, refactor)
- Design documents, GitHub integration...

Story states:

Unstarted

Started

Finished

Delivered

Accepted

Stories

("features")

can have

points

and tasks

be part of an

epic

or just a bug

or a chore

The screenshot displays the Pivotal Tracker interface with three main panels: **Current Iteration**, **Backlog**, and **Epics**.

- Current Iteration:** Shows a story titled "cleanup Convert Voucher models to strong parameters (AF)" with a "Finish" button. Below it, a story titled "enforce 'min & max per-txn purchase' for Redemptions" is shown with "Accept", "Reject", and "Delivered" buttons. The "TASKS (6/14)" section lists: "migration: add (min,max)_sales_per_txn, defaults to 1, 999 respectively", "add validations on min /max txn sales", and "UI field(s) to set min sales, defaults to 1".
- Backlog:** A list of stories, including "Nonticket product vouchers shouldn't be limited in quantity by total # seats remaining", "On Add Comps, #seats remaining should not account for zone restrictions", "add text field to walkup sales to allow tagging/identifying each sale for refund or transfer", "Imported by" not displayed correctly on ticket sales imports (AF), "On add/change redemptions, it's impossible to ONLY change redemption parameters without selecting one or more ticket types", "rs-import If reserved seating import would violate house cap, refuse to start import", "All-sales detail for a production should also include subscriber tickets (AF)", "customer order that came from 3rd party import should link to the import", "zone Allow editing seatmap shortname", "extern When same not-existing customer makes 2 separate external-vendor purchases for same showdate, A1 will try to 'create new customer' twice", "re-add UI for customer bulk import", "Easy way to 'clone' a redemption, eg for setting up multiple identical redemptions with distinct promo codes", and "landmark Allow limited HTML in show description and other user-facing fields that are customizable".
- Epics:** A list of epics, including "Boxoffice worker can import sales data from external vendors", "3rd party import w/reserved seating", "Seating Zones", "Cleanup NEEDED for Rails 5", "UI for editing seating zones", "Monthly recurring donations support", "Automatic 'reminder email' via Mailchimp API", "Detect and report email bounces (needs design; wait for someone to ask)", "Upgrade to Rails 5", "Should retail items be ALWAYS or NEVER associative with showdates?", "Mechanism for holding back seats", "Seatmaps test coverage", and "Season" as a first class abstraction.

Red arrows indicate the following connections:

- From "Unstarted" to the "Start" button on the "Nonticket product vouchers..." story in the Backlog.
- From "Started" to the "Start" button on the "On Add Comps, #seats remaining..." story in the Backlog.
- From "Finished" to the "Finish" button on the "Low hanging fruit stories done" story in the Backlog.
- From "Delivered" to the "Delivered" button on the "enforce 'min & max per-txn purchase' for Redemptions" story in the Current Iteration panel.
- From "Accepted" to the "Accept" button on the "enforce 'min & max per-txn purchase' for Redemptions" story in the Current Iteration panel.
- From "Stories" to the "rs-import" story in the Backlog.
- From "can have points" to the "2 Points" label on the "rs-import" story in the Backlog.
- From "and tasks" to the "TASKS (6/14)" section in the Current Iteration panel.
- From "be part of an epic" to the "Seating Zones" epic in the Epics panel.
- From "or just a bug" to the "Imported by" not displayed correctly story in the Backlog.
- From "or a chore" to the "cleanup Convert Voucher models..." story in the Current Iteration panel.

pivotaltracker.com

P&D vs. Agile: requirements & effort estimation

Tasks	In P&D	In Agile
Requirements elicitation	interviews, use cases	user stories
Requirements documentation	Software Requirements Specification, such as IEEE 830-1998 mandates	Cucumber scenarios and supporting documents (lo-fi sketches, etc.)
Change management	Version control for documentation and code	
Ensuring Requirements Features	Traceability to link features to code, tests, reviews, SRS...	Linking Pull Requests to story IDs in story-tracking system
Scheduling and Monitoring	Delivery date contracted based on cost estimates; use PERT charts, monitor milestones	Points, velocity
Cost estimation	Early in project, estimate cost based on manager experience or estimates of task size, combined with productivity metrics	Evaluate to pick range of effort for time & materials contract
Risk management (budget & schedule)	Early in project, identify top risks (%chance x impact) and actions to mitigate them	

Fallacies, Pitfalls, Concluding Remarks on BDD

ESaaS §7.10–7.11

© 2023 Armando Fox, David Patterson, Michael Ball
Licensed under Creative Commons Attribution-
NonCommercial-4.0 Unported License



Fallacies & pitfalls

- Using velocity to compare teams
- Assigning 3 (or more) points to stories whose complexity you're unsure of
- Conflating Finished, Delivered, Accepted
- Dividing work by layers vs. by stories

7. Stories vs. Layers



- “Dividing work by stories helps all team members understand app & be more confident when changing it”
- “Tracker helped us prioritize features and estimate difficulty”
- “We divided by layers [front-end vs. back-end vs. JavaScript, etc.] and it was hard to coordinate getting features to work”
- “It was hard to estimate if work was divided fairly...not sure if our ability to estimate difficulty improved over time or not”